

BLE (1) - 저전력 블루투스 (BLE) 란?

Updated: April 13, 2020

On this page

- 1. 저전력 블루투스
 - 1.1 기존 블루투스와의 차이점
 - 1.2 응용분야의 예

1. 저전력 블루투스 [Permalink](#)

저전력 블루투스(Bluetooth Low Energy, **BLE**)는 개인용 무선 네트워크 (Wireless Personal Area Network, **WPAN**) 기술의 일종으로, 블루투스라는 이름을 갖고 있지만 기존의 블루투스 (Bluetooth Classic, **BR/EDR**)와는 사실상 별개의 기술이다. 2010 년 출시된 **Bluetooth 4.0** 부터 저전력 (Low Energy, **LE**) 프로토콜을 지원하기 시작했고, 2016 년 **Bluetooth 5.0** 출시와 함께 그 기능이 보다 확장되었다.

Terms	Details	Standard	Time
BR	Basic Rate (1 Mbps)	1.0 – 1.2	1999 – 2003
EDR	Enhanced Data Rate (2 – 3 Mbps)	2.0	2004
HS	High speed	3.0	2009
LE	Low Energy (1 Mbps), super saving energy	4.0	2010
	LE, support LTE	4.1	2013
	LE, allow connect IP, security and high speed	4.2	2014
	2x speed / 4x range / 8x broadcast capability	5.0	2016

출처: <https://www.semiconductorstore.com/blog/2017/Bluetooth-5-versus-Bluetooth-4-2-whats-the-difference/2080>

1.1 기존 블루투스와의 차이점 [Permalink](#)

기존의 블루투스 (**BR/EDR + HS**) 기술이 데이터 전송 속도 (**data rate**)에 초점을 맞추고 있는 반면, BLE 는 전력 소모를 줄이는 것에 초점을 맞추며 헬스케어, 피트니스, 보안 시스템 등의 특정 응용분야 (**application**)에 적합한 기술로 대두되었다.

BLE 는 대량의 정보를 빠르게 전송하는 것보다는 **소량의 정보를 주기적으로** 또는 간헐적으로 보내는 시스템에 적합한 기술이다.

1.2 응용분야의 예 [Permalink](#)

무선 센서 네트워크 (**WSN**)



출처: <https://www.e-zigurat.com/innovation-school/blog/what-do-the-next-five-years-hold-for-the-iot>

무선 센서 네트워크 (Wireless Sensor Network, **WSN**)는 BLE 를 적용하기에 적절한 응용분야이다. 예를 들어, 특정 공간에 다수의 센서를 배치하고 각 센서의 정보를 수집한다고 해보자. 이러한 응용분야에서 센서는 소량의 정보만을 전송하며, 실시간으로 각 데이터를 수집할 필요 또한 없으므로 시스템 구축에 있어 속도에 제약을 받지 않는다.

이와 유사한 응용분야의 경우 데이터 전송 속도보다, 적은 용량의 배터리를 가지고 얼마나 오래 시스템을 구동할 수 있는지가 중요한 이슈라고 할 수 있다.

헬스케어 (Healthcare)

[Home · iOS & Swift Tutorials](#)

Core Bluetooth Tutorial for iOS: Heart Rate Monitor

In this Core Bluetooth tutorial, you'll learn how to discover, connect to, and retrieve data from compatible devices like a chest-worn heart rate sensor.



출처:

<https://www.raywenderlich.com/231-core-bluetooth-tutorial-for-ios-heart-rate-monitor>

사용자의 건강 상태를 모니터링하는 **헬스케어** 응용분야 또한 **BLE** 를 이용하기 적합한 모델이라고 할 수 있다. 예를 들어, 심박수 측정 센서를 이용해 사용자의 심박수를 측정하고, 해당 정보를 **APP** 으로 전송하는 어플리케이션을 개발한다고 해보자.

만약, 심박수 측정 센서를 통해 측정한 값이 **110 BPM** 일 경우, **BLE** 프로세서에서 실질적으로 보내야 하는 **110** 이라는 데이터는 1 바이트 (byte) 만으로 표현 가능하며, 이상이 있는 경우 (심박수치가 너무 높거나 낮은 경우)가 아니라면 실시간으로 데이터를 전송할 필요도 없을 것이다.

따라서, 이러한 경우에도 **BLE** 를 이용하여 효율적인 저전력 시스템 설계가 가능하다.

Reference

<https://www.semiconductorstore.com/blog/2017/Bluetooth-5-versus-Bluetooth-4-2-whats-the-difference/2080>

<https://www.raywenderlich.com/231-core-bluetooth-tutorial-for-ios-heart-rate-monitor>

BLE (2) - BLE 프로토콜 스택

Updated: April 14, 2020

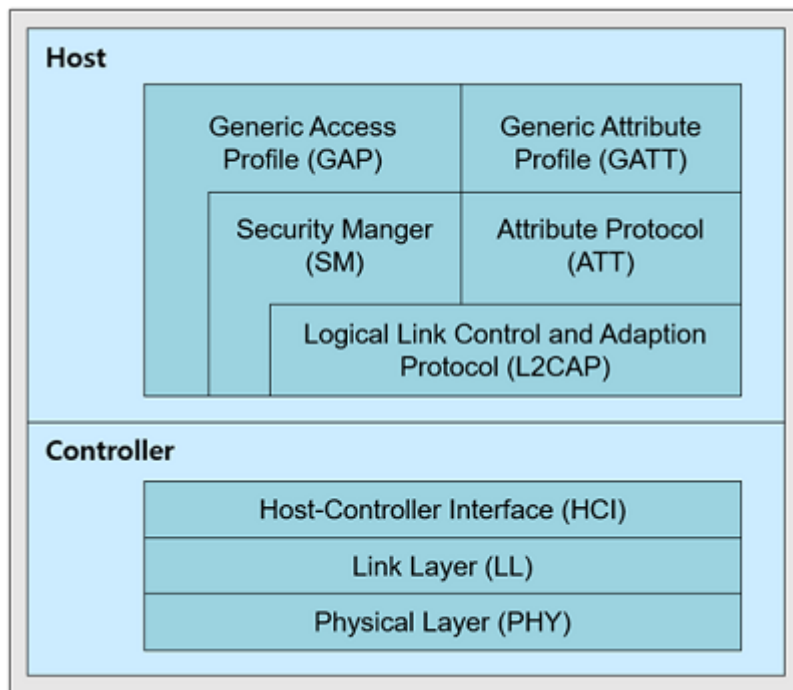
On this page

- 2. BLE 프로토콜 스택
 - 2.1 Physical layer (PHY)
 - 2.2 Link layer (LL)
 - 2.3 HCI, L2CAP, SM
 - 2.3.1 HCI
 - 2.3.2 L2CAP
 - 2.3.3 SM
 - 2.4 Attribute Protocol (ATT)
 - 2.5 Generic Access Profile (GAP)

2. BLE 프로토콜 스택 [Permalink](#)

BLE 프로토콜 스택 (**BLE protocol stack, BLE stack**)은 BLE 디바이스의 구조를 보여주며, 이는 블루투스 표준 (**Bluetooth Core Specification**)에서 정의하고 있는 LE (**Low Energy**) 모드로 동작하는 디바이스가 갖춰야하는 기능들을 포함하고 있다.

The BLE protocol stack implements all the mandatory and optional features of LE mode compliant to Core Specification



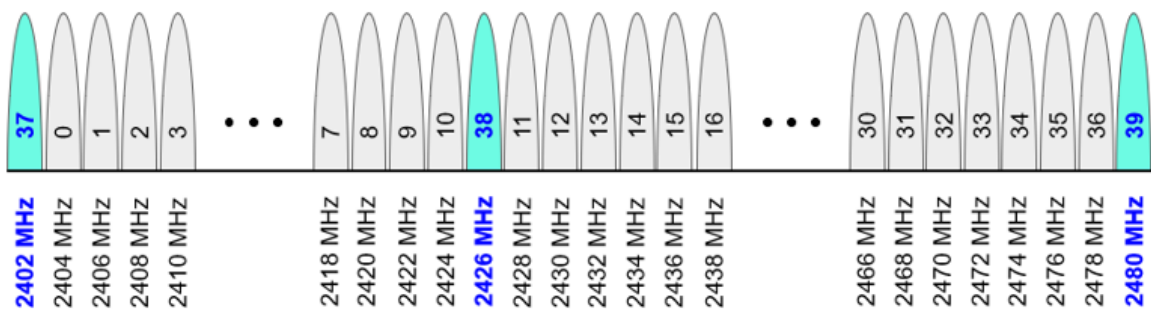
BLE 스택은 크게 **Host** 와 **Controller** 로 구성되어 있고, 각 영역은 다양한 종류의 레이어 (**layer**) 혹은 프로파일 (**profile**)로 이루어져있다. BLE 디바이스는 각 레이어에서 특정 상태(**state**) 혹은 역할(**role**)을 가지고 동작하게 되며, BLE 시스템을 개발하기전 BLE 스택의 각 요소가 어떤 역할을 하는지를 이해하는 것은 필수이다.

Notice: 다음의 서술하는 내용에서는 혼돈을 방지하기 위해 영어로 표기된 고유 명사의 대부분을 원문 영단어 그대로 사용합니다. 또한, 필자가 이해하고 있는 방식으로 바탕으로 서술하고 있으므로 용어나 표현에 있어서 잘못 서술하는 내용이 있을 수 있습니다.

2.1 Physical layer (PHY)Permalink

PHY 는 RF 시스템 및 무선 전파(**radio**) 신호를 송수신하기 위한 **하드웨어**로 구성된다. BLE 디바이스는 2.4 GHz 의 **ISM** 주파수 대역에서 동작하며, **GFSK** 과 **FHSS** 방식으로 데이터를 송수신한다.

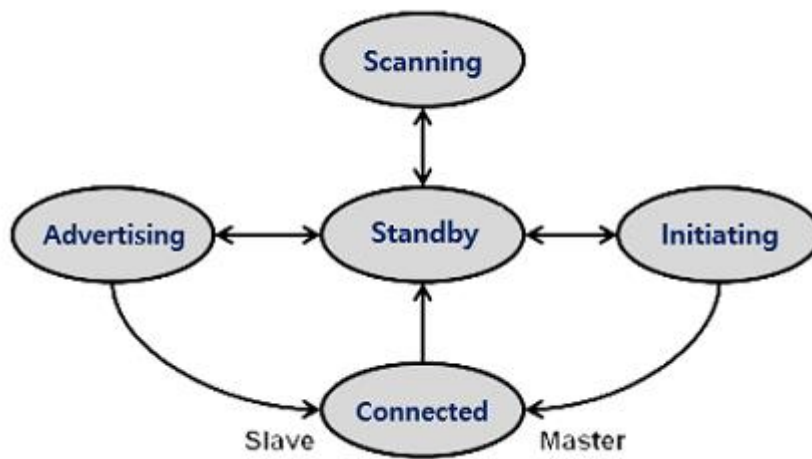
PHY 에서 BLE 시스템은 2.4 GHz 주파수 영역의 일부를 40 개의 구간으로 나누어 신호를 주고 받으며, 각각의 영역을 **채널**이라고 부른다. 각 채널은 중심 주파수를 기준으로 2 MHz 의 대역폭을 가지며, 할당된 40 개의 채널 중 37-39 번 채널은 **advertising** 채널로 사용되고, 나머지 37 개의 채널은 **data** 채널로 사용된다 (각 채널에 대해서는 차차 알게 될 것이다).



위 그림에서 확인할 수 있듯이 중심 주파수 크기와 채널 번호 순서는 서로 동일하지 않으며, 일부 BLE 시스템의 경우는 2.402~2.48 GHz 대역 대신에 2.4~2.4835 GHz 대역을 사용하기도 한다.

2.2 Link layer (LL)Permalink

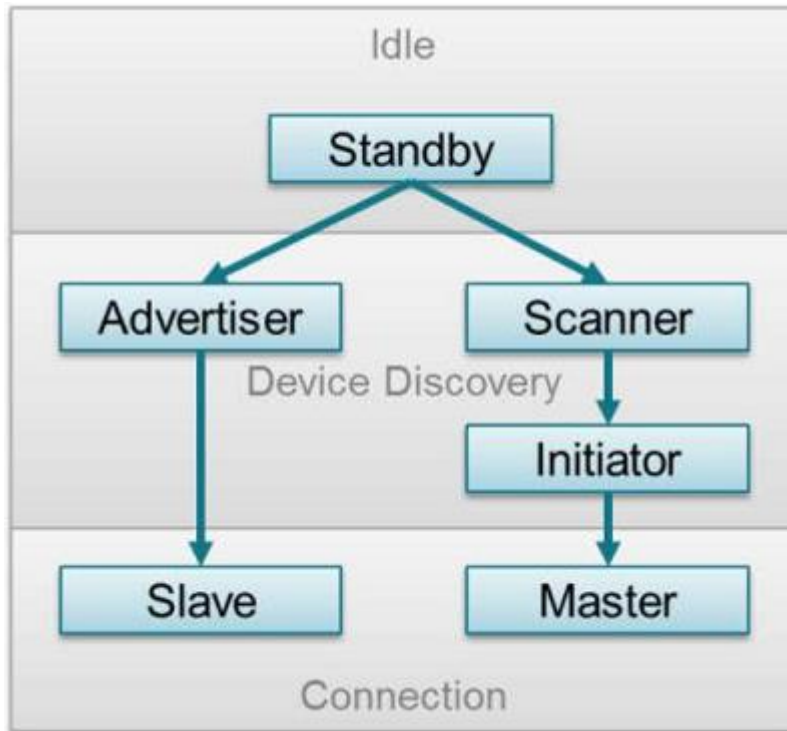
LL 는 **PHY** 와 직접적으로 상호작용하는 레이어이고, 하드웨어와 소프트웨어 사이에서 동작한다. 하드웨어와 연결되어 BLE 디바이스 간의 연결을 직접적으로 관리하는 레이어로 암호화 (**encryption**), 연결 상태 및 채널 업데이트 (**connection or channel update**) 등의 역할을 수행한다.



위 그림에서와 같이 BLE 디바이스는 LL 에서 다음 중 하나의 상태로 동작한다.

- Standby
- Advertising
- Scanning
- Initiating
- Connected

두 개의 BLE 디바이스가 연결을 시도할 때, 각각의 디바이스는 LL 에서 다음의 그림과 같이 동작한다.



출처:

https://dev.ti.com/tirex/content/simplelink_cc2640r2_sdk_1_40_00_45/docs/blestack/ble_user_guide/html/ble-stack-3.x/gap.html

먼저 BLE 연결을 시작하고자 하는 **master** 디바이스 (오른쪽)에서는 **Scanning** 동작을 수행한다. **Scanning**이란, 단어 뜻 그대로 주변에 BLE 연결이 가능한 디바이스가 있는지 확인하는 과정이다. 다음으로 BLE 연결을 대기하는 **slave** 디바이스의 경우 (왼쪽) **Advertising**이란 동작을 통해 해당 디바이스가 근처에 있다는 것을 주변에 알린다. 이후, **master** 디바이스에서 **Scanning**한 디바이스 중 하나에 연결을 시도할 경우, 해당 디바이스는 **LL**에서 연결을 준비하는 **initiating** 상태에 진입하게 되고, 무선 연결에 성공하고나면 **master**와 **slave** 디바이스는 모두 **Connected** 상태에 놓이게 된다.

이러한 BLE 연결 과정은 이후 설명할 **GAP**에 의해 관리된다.

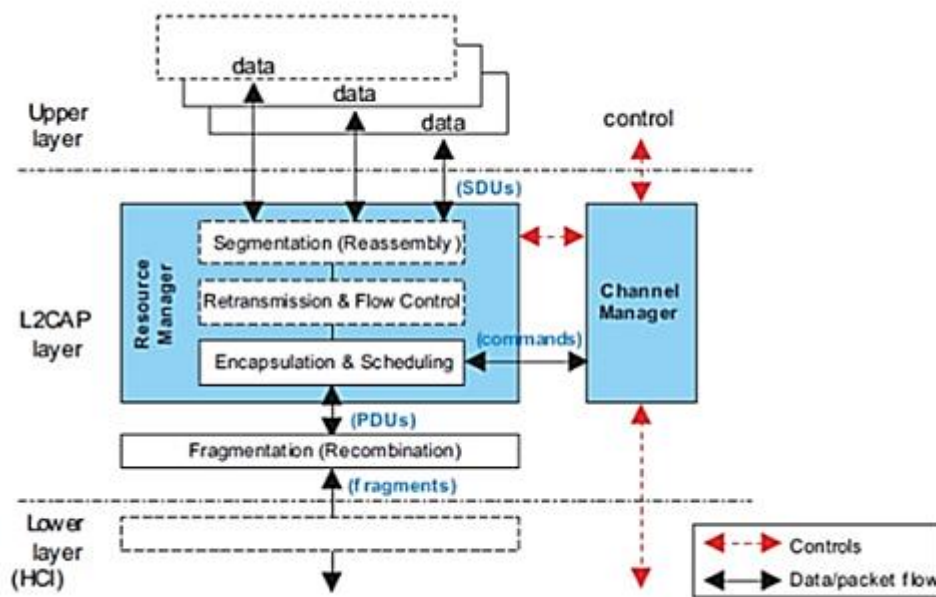
2.3 HCI, L2CAP, SMP, Permalink

2.3.1 HCI Permalink

HCI (Host-Controller Interface)는 **Host**와 **Controller**가 분리되어 있는 경우 두 모듈 사이의 상호작용을 담당하는 레이어이다. 앞으로 소개할 **Nordic Semiconductor**사의 **BLE** 프로세서를 비롯해, 대부분의 **BLE** 디바이스는 **Host**와 **Controller**가 단일 칩으로 구성되어 있다.

2.3.2 L2CAP Permalink

L2CAP는 **Logical Link Control and Adaptation Protocol**의 약자로, 상위 레이어 (**GATT/GAP**)와 하위 레이어 사이의 상호작용을 담당한다. **LL**에서 만들어진 **BLE** 패킷 (**Packet**)에 실제 어플리케이션 단에서 사용되는 데이터를 포함시키거나 추출하는 역할 등을 수행한다.



출처:

https://dev.ti.com/tirex/content/simplelink_cc26x2_sdk_1_60_00_43/docs/ble5stack/ble_user_guide/html/ble-stack-5.x/l2cap.html

2.3.3 SMP

Security Manager 는 자주 연결하는 디바이스 (**peer**) 사이의 보안 코드 (**key**)를 관리하고 배포하는 역할을 수행한다.

2.4 Attribute Protocol (ATT)

ATT 는 서버 (**server**)와 클라이언트 (**client**) 사이의 **데이터 교환에 대한 규칙**으로, 무선으로 연결된 BLE 디바이스 사이의 데이터 교환은 **ATT** 에 의해 이뤄진다. 이때, 연결된 디바이스 중 센서의 값이나 스위치의 상태, 위치에 대한 정보 등을 가지고 있는 디바이스를 **GATT server** 라고 부르고, 해당 정보를 요청하는 디바이스를 **GATT client** 라고 부른다.

또한, **GATT** 는 Generic Attribute Profile 의 약자로 **ATT** 를 기반으로 주고받는 **데이터의 구조**를 정의한다. **ATT** 와 **GATT** 에 대해서는 다음 포스트에서 보다 구체적으로 다룰 예정이다.

2.5 Generic Access Profile (GAP)

GAP 는 연결 및 advertising 동작을 관리하는 최상위 레이어로 각 디바이스의 **LL** 상태를 결정하고, 서로 다른 BLE 디바이스 사이의 상호작용을 관리하는 모듈이다. BLE 디바이스는 **GAP** 에 대해 다음 중 하나의 역할 (**role**) 로 동작한다.

- Broadcaster
- Observer
- Peripheral

- **Central**

먼저, **Peripheral** (주변기기)로 동작하는 디바이스는 **LL** 에서 **Advertising** 상태를 유지하고 있으며, 다른 디바이스로부터의 연결 요청을 기다린다. 앞서 언급했듯이 **Advertising** 이란, 주변의 **BLE** 디바이스에 자신의 정보를 알리는 동작이고, 일반적으로 이 과정에서는 해당 디바이스의 이름과 간단한 정보만을 송신한다. 예를 들어, 아래의 그림과 같이 스마트폰에서 무선 이어폰과 연결하기 위해 근처 블루투스 기기를 찾고(**Scanning**) 있다고 가정해보자. 이때 본인의 이어폰 (e.g. AirPods)을 포함한 다수의 블루투스 기기의 이름이 보일텐데, 이 때 감지되는 디바이스들은 모두 **Advertising** 상태에 있는 **Peripheral** 디바이스라고 할 수 있다.



출처: <https://www.semiconductorstore.com/blog/2017/Bluetooth-5-versus-Bluetooth-4-2-whats-the-difference/2080>

위 예시에서의 스마트폰과 같이 **Peripheral** 디바이스를 **Scanning** 하고, 그 중 하나의 디바이스와 연결까지 가능한 디바이스는 **GAP** 에서 **Central** (중앙장치)로서 동작하는 디바이스이다.

BLE 연결은 Central 과 Peripheral 디바이스 사이에서 이뤄지고, Central 디바이스에 의해 시작된다.

반면에 **Peripheral** 디바이스가 연결 과정 없이 **Advertising** 동작만을 수행하는 경우, 이러한 디바이스를 **Broadcaster** 라고 한다. 기회가 되면 설명하겠지만, **Broadcaster** 로 동작하는 디바이스는 **Central** 디바이스에서도 연결할 수 없으며, **Advertising** 데이터에 자신 이름 이외에도 실질적인 데이터를 실어서 전송한다.

일반적으로 이러한 장치를 비콘(Beacon)이라고 부른다.

또한, **Central** 디바이스가 연결 과정 없이 **Scanning** 동작만을 수행하는 경우 해당 디바이스를 **Observer** 라고 부르며, **Observer** 의 경우 일반적으로 **Advertising** 패킷 내의 데이터를 타겟으로 하므로 **Beacon** 신호를 수신하고자 할 때 사용한다.

Reference

https://dev.ti.com/tirex/content/simplelink_cc2640r2_sdk_1_40_00_45/docs/blestack/ble_user_guide/html/ble-stack-3.x/gap.html

https://dev.ti.com/tirex/content/simplelink_cc26x2_sdk_1_60_00_43/docs/ble5stack/ble_user_guide/html/ble-stack-5.x/l2cap.html

<https://www.semiconductorstore.com/blog/2017/Bluetooth-5-versus-Bluetooth-4-2-whats-the-difference/2080>

BLE (3) - ATT/GATT 이해하기

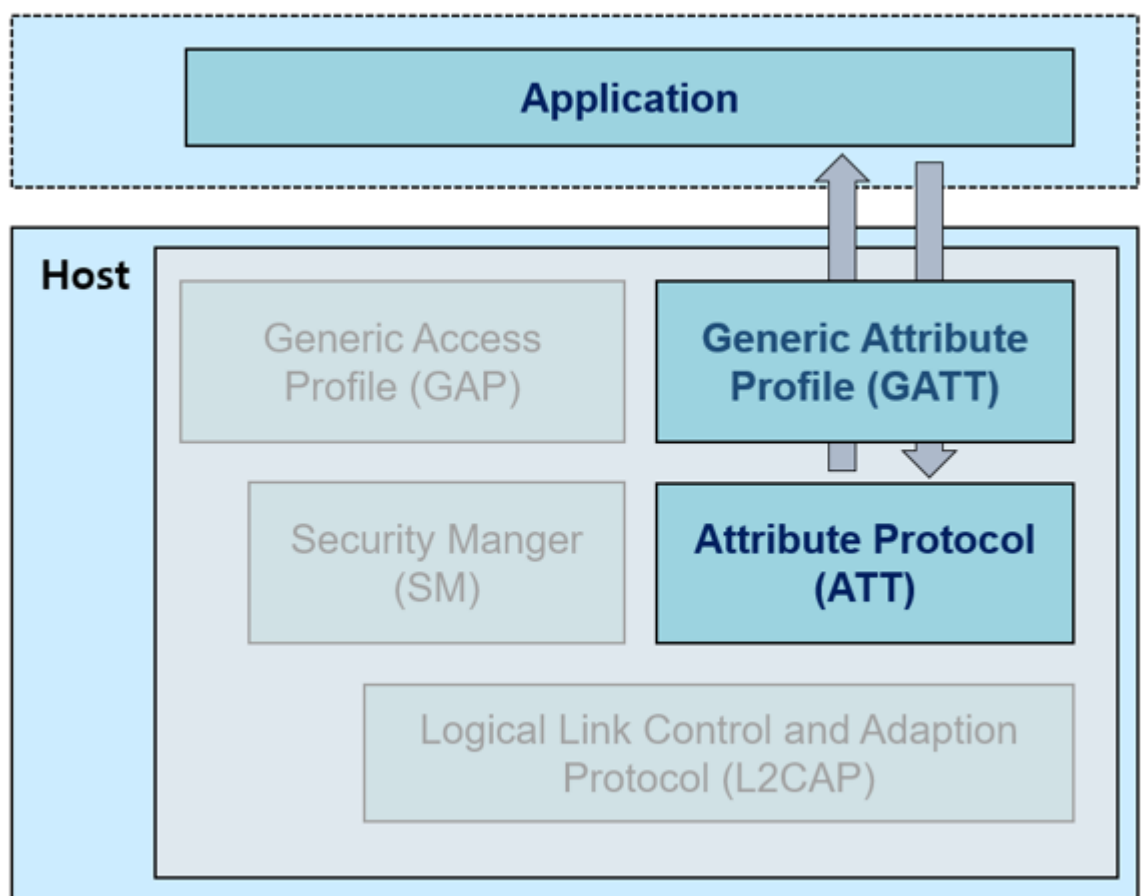
Updated: April 16, 2020

On this page

- 3. ATT/GATT
 - 3.1 GATT
 - 3.2 Attribute Table 예시
 - 3.3 Service 정의 순서
 - 3.4 GATT Server/Client

3. ATT/GATTPermalink

BLE 프로토콜 스택에서 Attribute protocol (ATT)은 서버 (server)와 클라이언트 (client) 사이의 데이터 교환에 대한 규칙을 정의한다. 다음 그림에 묘사된 것 같이, 어플리케이션 단에서의 데이터 교환은 ATT를 기반으로 이뤄지며 각각의 데이터는 구조는 Generic Attribute Profile (GATT)에 의해 정의되는 데이터 구조를 따른다.

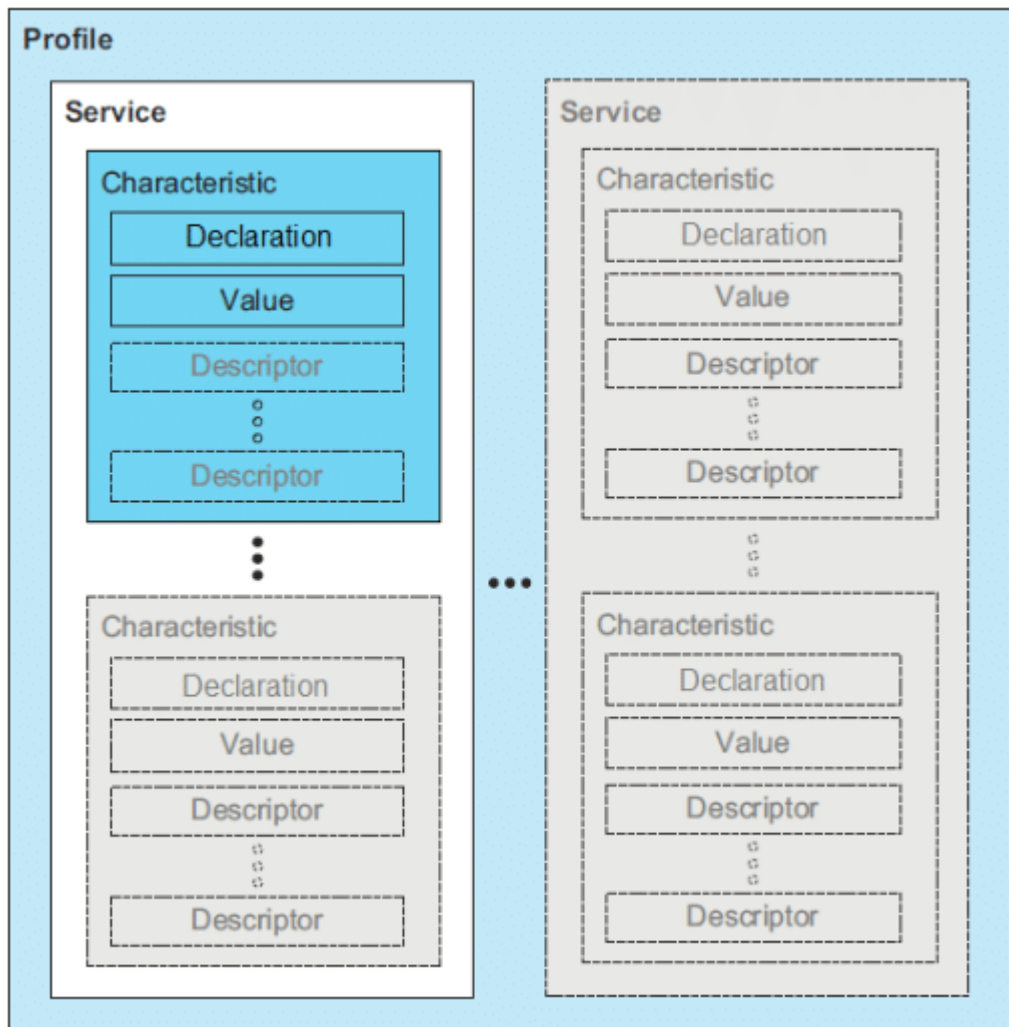


대부분의 BLE 칩 제작사 (e.g. Nordic Semiconductor, Texas Instruments, .etc)는 BLE 프로토콜 스택 소스를 제공하기 때문에, 펌웨어 개발자가 직접 BLE 스택을 프로그래밍하거나 수정하는 경우는 드물다. **다만, BLE 디바이스의 데이터가 어떤 식으로 생성되고, 어떤 정보를 기반으로 데이터를 교환하는지 알아두는 것은 전반적으로 BLE 프로토콜을 이해하는데 큰 도움이 될 것이다.**

3.1 GATTPermalink

GATT 에 의해 정의되는 BLE 시스템의 데이터 구조는 다음의 그림과 같이 **service** 와 **characteristic** 으로 표현된다.

Service is a collection of information, and **Characteristic** is a set of actual values and information



출처:

https://www.researchgate.net/figure/BLE-Data-Hierarchy_fig2_317553032

Service 는 특정 어플리케이션을 구현하기 위해 필요한 데이터의 집합이라고 할 수 있으며, 각각의 데이터는 **characteristic** 에 의해 정의된다. 예를 들어, 심박수 측정 데이터를 관리하기 위한 **service** 를 정의할 경우, 해당 **service** 는 실제 심박 수치를 저장하는 **characteristic** 을 비롯해, 센서의

단위나 모델 번호, 또는 보안 관련 파라미터 등의 정보를 관리하는 다수의 **characteristic** 를 포함할 수 있다.

이러한 **service** 와 **characteristic** 에 대한 정보는 **attribute** 라는 최소 데이터 유닛에 의해 정의되고, BLE 디바이스 내의 **attribute** 에 대한 정보는 최종적으로 **attribute table** 내에 저장된다.

각각의 **attribute** 는 다음의 세 가지 정보를 포함하고 있다.

- type (UUID)
- handle
- permission

3.2 Attribute Table 예시 [Permalink](#)

다음은 **attribute table** 의 예시를 보여주며, 각 행(row)은 하나의 **attribute** 를 나타낸다.

Attribute Table				
Type	Handle	UUID	Permission	Value
Service declaration	0x000C	0x2800 (Service declaration)	Read Only, No Authentication No Authorization	0x180D (HRS)
Char. dec.	0x000D	0x2803 (Char. dec.)	Read Only, No Authentication No Authorization	Char. Info. (... UUID: 2A37, value handle: 0x000E, ...)
Char. val.	0x000E	0x2A37 (Spec. char. value)		167 Beats per min.
Char. dec.	0x000F	0x2803		

handle 은 **attribute** 의 주소 값에 해당되고, permission 에는 단어 뜻 그대로 해당 **service** 또는 **characteristic** 의 접근 권한에 대한 정보 등이 저장된다. UUID 는 **Universally Unique ID** 의 약자로, **GATT** 데이터 성분을 구분하기 위한 고유 식별자로 사용된다.

3.3 Service 정의 순서 [Permalink](#)

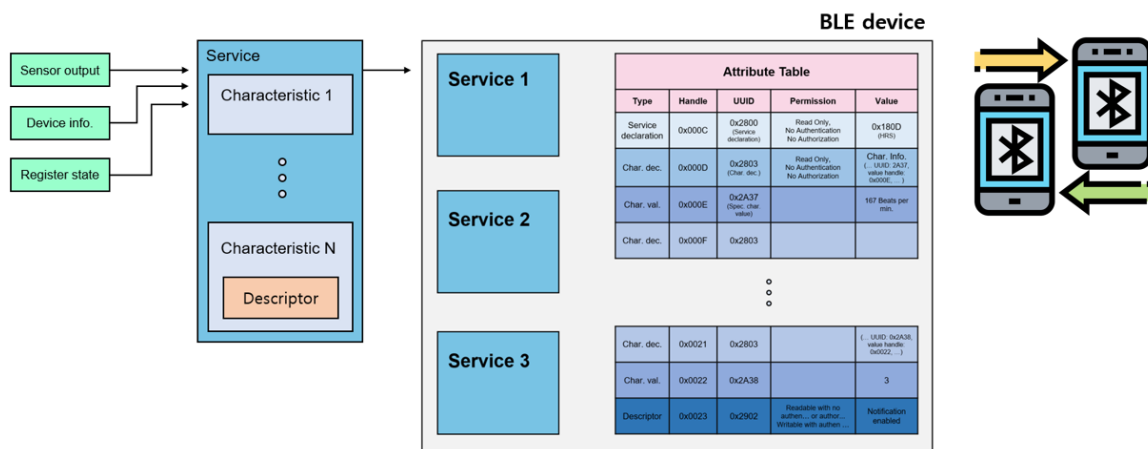
0x2800 은 **service** 선언에 대한 **attribute UUID** 값이다. 위의 **attribute table** 그림에서 맨 위쪽 행을 보면, **UUID = 0x2800** 값을 갖는 **attribute** 를 이용해 **service** 를 선언하고 있다. 본 예시에서 해당 **attribute** 의 **value** 에는 심박(Heart Rate) 측정 **service** 에 대한 **UUID** 값 0x180D 이 저장되어 있음을 볼 수 있다.

Bluetooth SIG에서는 자주 사용되는 **service** 에 대한 **UUID** 를 사전에 정의해두고 있으며, 이러한 **service** 의 경우 **16 bits** 사이즈의 **UUID** 값을 갖는다. 사전에 정의된 **GATT service** 에 대한 **UUID** 값은 [본 링크](#)에서 확인 가능하다.

다음으로, 0x2803 은 **characteristic** 선언에 대한 **attribute UUID** 값이고, 해당 **attribute** 의 **value** 에는 선언하고자 하는 **characteristic** 의 **UUID = 0x2A37** 와 **handle** 값이 저장되어 있다. 0x2A37 은 **HRS** 에서 심박 수 센서 값을 저장하는 **characteristic** 의 **UUID** 이고, 위의 예시에서 볼 수 있듯이 **UUID = 0x2A37** 값을 가지는 **attribute** 의 **value** 에는 167 이란 정보가 저장되어 있는 것을 볼 수 있다.

BLE 디바이스는 하나 이상의 **service** 를 포함할 수 있으며, 각각의 **service** 또한 하나 이상의 **characteristic** 을 포함할 수 있다.

다음은 **BLE 디바이스** 내부의 데이터 구조의 예시를 보여준다.



3.4 GATT Server/Client [Permalink](#)

무선 연결된 **BLE 디바이스** 중 데이터를 가지고 있는 디바이스가 **GATT server** 가 되고, 데이터를 요청하는 디바이스가 **GATT client** 가 된다.

예를 들어, 스마트폰 APP 을 이용해서 **BLE** 센서 노드 (e.g. 센서 데이터 수집 후, **BLE** 프로세서를 기반으로 데이터를 송신하는 디바이스)의 데이터를 수신하고자 할 때, **BLE** 통신을 시작하는 디바이스는 스마트폰이기 때문에 스마트폰이 **GAP central** 이 되고, 센서 노드는 **GAP peripheral** 이 된다. 그리고 실질적인 데이터를 가지고 있는 **BLE** 센서 노드는 **GATT server** 가 되고, 데이터를 요청하는 스마트폰은 **GATT client** 가 된다.

Reference

https://www.researchgate.net/figure/BLE-Data-Hierarchy_fig2_317553032

<https://devzone.nordicsemi.com/nordic/short-range-guides/b/bluetooth-low-energy/posts/ble-characteristics-a-beginners-tutorial>

BLE (4) - BLE 디바이스는 어떻게 연결할까?

Updated: April 22, 2020

On this page

- 4. BLE 무선연결 과정
 - 4.1 Advertising
 - 4.2 Scanning
 - 4.3 Connection

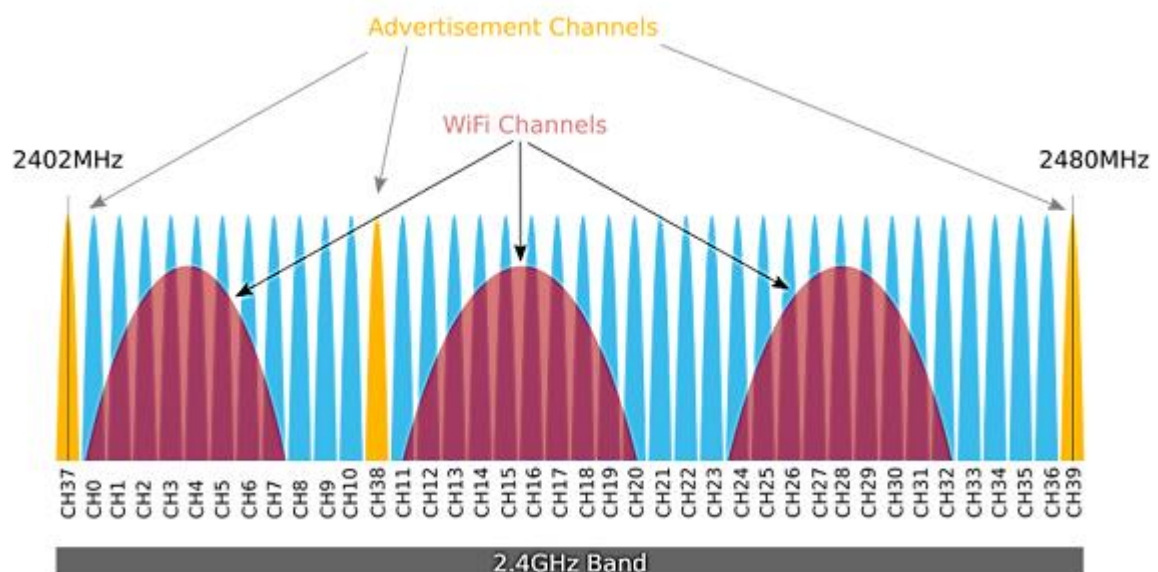
4. BLE 무선연결 과정 Permalink

이전 포스트에서 BLE 시스템의 연결 및 advertising 동작을 관리하는 최상위 레이어는 **GAP** (Generic Access Profile)이고, 서로 다른 BLE 디바이스 사이의 상호작용을 관리하는 모듈이라고 소개한 바 있다 (**BLE 프로토콜 스택**). 이번 포스트에서는 **BLE 연결 과정**에 대해 조금 더 자세하게 소개하고자 한다.

4.1 Advertising Permalink

BLE 무선연결의 시작은 **GAP peripheral** 로 동작하는 디바이스에서 advertising 동작을 수행하는 것으로 시작된다. **Advertising 이란**, 해당 디바이스의 정보 (**Device Name, Mac Address, Tx Power, .etc**)를 **GAP central** 로 동작하는 근처 BLE 디바이스에 알리는 동작이다.

아래 그림과 같이 advertising 을 수행하는 디바이스는 **PHY** (Physical Layer)의 40 개의 채널 중 3 개의 advertising 채널(37 번-39 번)을 이용하며, 각 채널은 2.4 GHz 대역 중에서도 WiFi 와의 간섭이 가장 적을 것으로 예상되는 주파수를 중심 주파수로 갖는다.

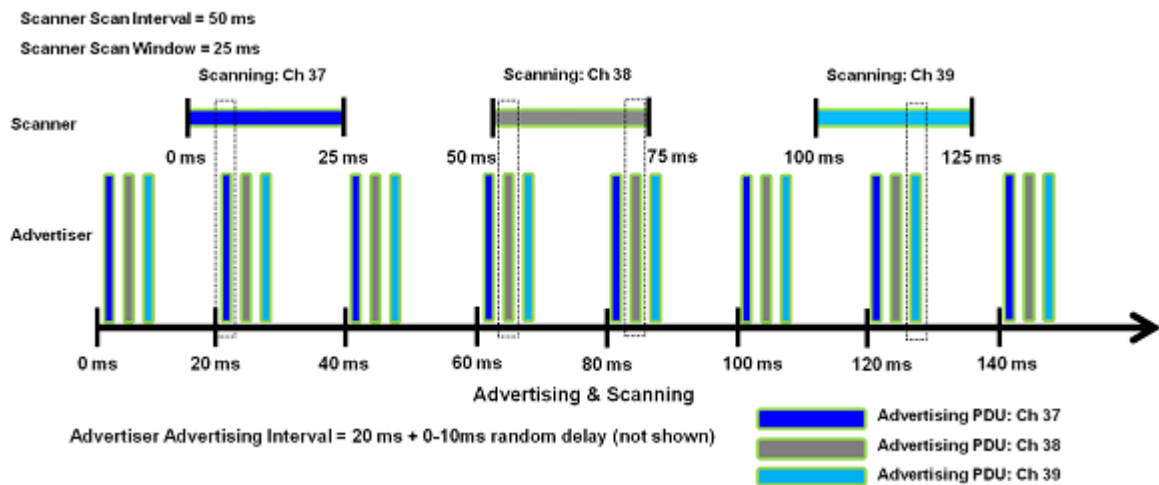


출처: <https://www.argenox.com/library/bluetooth-low-energy/ble-advertising-primer/>

Advertising interval: 20 ms ~ 10.24 secs.

4.2 ScanningPermalink

다음으로, **GAP central** 로 동작하는 디바이스는 BLE 무선연결을 위해 근처에 연결 가능한 디바이스가 있는지 찾아본다 (**scanning**). 이는 즉, 근처에 advertising 하고 있는 디바이스가 있는지를 확인하는 작업이므로, scanning 또한 40 개의 채널 중 advertising 채널에 대해 수행하게 된다.



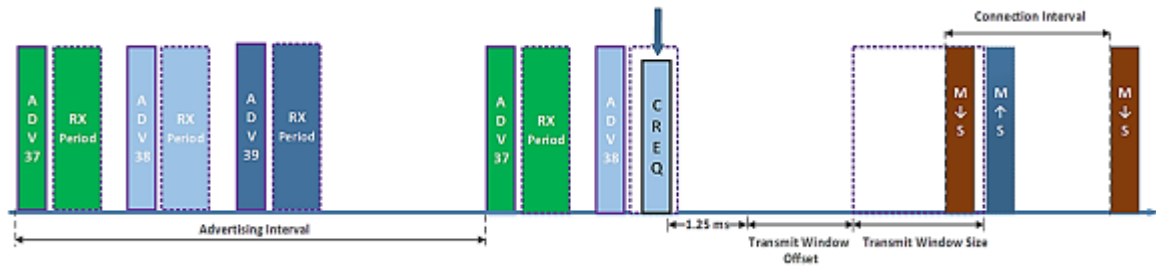
출처: <https://microchipdeveloper.com/wireless:ble-link-layer-discovery>

위 그림에서 볼 수 있듯이, 일반적으로 **Advertiser = peripheral** 디바이스는 한 번에 3 개의 채널로 advertising 하고, **Scanner = central** 디바이스는 일정 간격으로 하나의 채널을 scanning 한다. 주요 scanning 파라미터로는 **scan interval** 과 **scan window** 가 있는데, 그림에서 볼 수 있듯이 **scan interval** 은 scanning 시작시간 사이의 간격을 정의하고, **scan window** 는 한 채널에서 실제로 scanning 하는 시간 (구간)을 정의하고 있다.

advertising 채널과 scanning 채널이 일치하는 경우에만 Scanner 에서 advertising 데이터를 수신할 수 있으며, advertising 시간은 수 ms 인 반면 Scanner 에서는 수십에서 수백 ms 동안 scanning 동작을 수행한다. 따라서, Scanner 에서는 기본적으로 Advertiser 에 비해 보다 많은 양의 전류를 소비하게 된다.

4.3 ConnectionPermalink

다음의 그림은 **peripheral** 디바이스에서의 BLE 연결 과정을 보여준다.

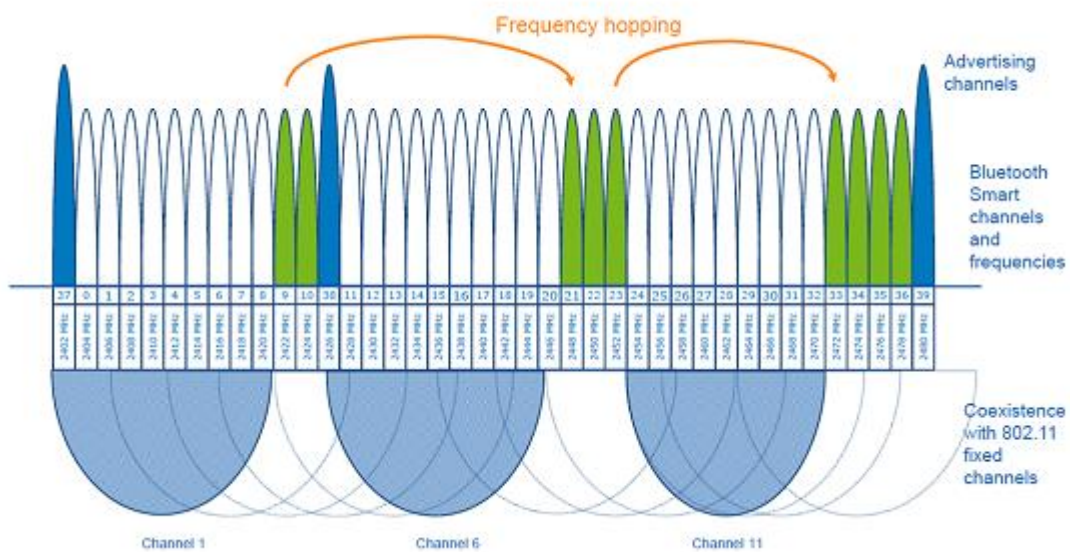


출처: <https://devzone.nordicsemi.com/nordic/short-range-guides/b/bluetooth-low-energy/posts/bluetooth-smart-and-the-nordics-softdevices-part-2>

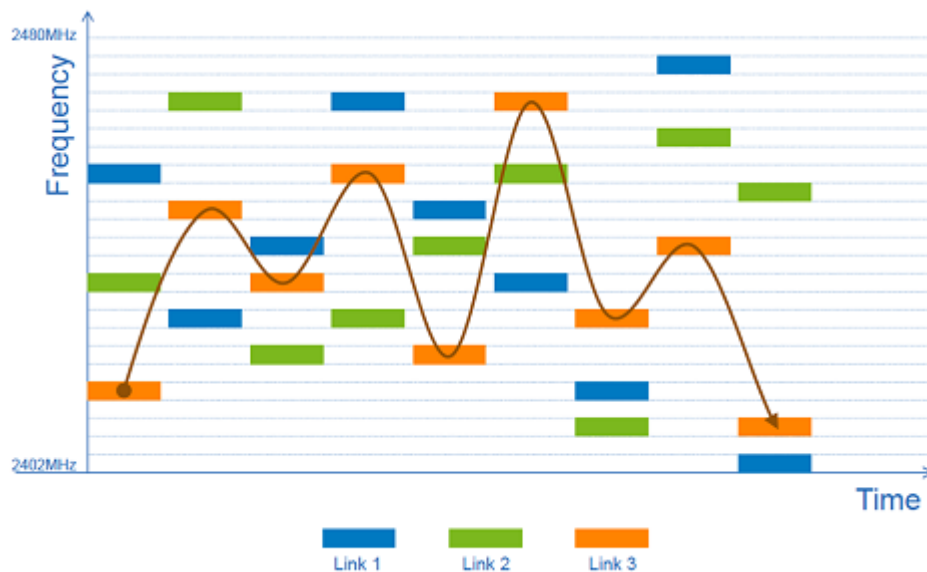
앞서 BLE 무선연결은 **peripheral** 디바이스에서 **advertising** 동작을 수행하는 것으로 시작된다고 언급한 바 있는데, 실제로 연결 가능한 **peripheral** 디바이스는 위 그림에서처럼 **advertising** 직후 잠깐동안 해당 채널의 신호를 수신하는 시간을 갖는다. 그림에서 볼 수 있듯이 **peripheral** 디바이스에서 **advertising** 주기와 **scanning** 주기가 겹치는 시점에 해당 채널(38 번)을 통해 **central** 디바이스의 **연결 요청 신호 (CREQ: connection request)**를 수신하게 되면, 두 디바이스 사이의 무선연결이 시작된다.

무선연결이 시작되고나면, 데이터를 교환에 앞서 연결 파라미터를 주고 받는다. BLE에서는 통신 과정에서 **advertising** 채널을 제외한 37 개의 채널을 이용해 **FHSS** 방식으로 데이터를 주고 받는다. 즉, 중심 주파수 (**frequency**)를 바꿔가면서(**hopping**) 데이터를 주고 받는 것인데, 처음 연결시 동기화 (**synchronization**) 작업이 필요하지만, 방해 전파나 잡음의 간섭을 줄일 수 있으며, 호핑 코드 (**hopping code**)만 다르면, 같은 공간 내에서도 무선 디바이스 사이의 간섭을 최소화할 수 있다는 이점이 있다.

hopping code: Frequency Hopping Spread Spectrum 통신 기술에서 어떤 순서로 주파수 대역을 변경할 것인지에 대한 정보.



출처: <https://microchipdeveloper.com/wireless:ble-link-layer-channels>



출처: <https://microchipdeveloper.com/wireless:ble-link-layer-channels>

BLE 연결 직후에는 호핑 코드와 같이 하드웨어 단에서 필요한 파라미터를 포함해 무선연결과 관련된 파라미터를 교환하게 된다. 또한, 이 시점에 연결 주기, 타임아웃 시간 (*일정 시간 통신이 되지 않을 경우 연결이 끊어졌다고 판단하는 시간*) 등의 정보를 주고 받으며, 이러한 파라미터는 대부분 **central** 디바이스에서 의해 결정된다.

무선연결 파라미터를 정하는데 있어서 우선적인 결정 권한은 **central** 디바이스가 가지고 있지만, **peripheral** 디바이스에서도 다음의 연결 파라미터를 **제안**할 수 있다.

- minimum connection interval
- maximum connection interval
- slave latency
- connection supervision timerout

Connection interval: **7.5 ms ~ 4 secs.**

Reference

<https://microchipdeveloper.com/wireless:ble-link-layer-channels>

<https://microchipdeveloper.com/wireless:ble-link-layer-discovery>

<https://devzone.nordicsemi.com/f/nordic-q-a/28058/what-is-connection-parameters>

<https://devzone.nordicsemi.com/nordic/short-range-guides/b/bluetooth-low-energy/posts/bluetooth-smart-and-the-nordics-softdevices-part-2>

<https://www.argenox.com/library/bluetooth-low-energy/ble-advertising-primer/>